Product Requirements Document: BinaryRepair MVP

Overview

Full title: AI-Driven Automated Binary Vulnerability Repair and Correctness Verification
Version: 1.0
Created: 15 October 2026
Status: Confirmed product brief; technical design pending
Owner: Five-person student project team

Problem: A software company has vulnerable binary programs without original source code. It needs a way to generate repairs directly from binaries and assess whether the target unsafe behavior is blocked while required normal behavior is preserved.

MVP goal: Demonstrate two AI-assisted repair routes on the same public stack-buffer-overflow test case compiled as Linux x86-64 ELF, producing two separate patched binaries and comparable validation evidence through a graphical interface.

Delivery: Proposal → MVP → final application. The proposal is due one week from 15 October 2026, approximately 22 October; exact submission time is TBD. MVP and final-app dates are TBD. The proposal deadline is not the MVP deadline.

Research basis: research-BinaryRepair.md, dated 15 October 2026. It contains sourced documentation/literature research, not completed implementation experiments. No binary has been repaired or verified as part of this PRD.

Decision precedence: This PRD incorporates the later confirmed interview decisions. These supersede the report's earlier CLI recommendation, unresolved same-sample question, and strict in-place option. The research file remains a dated record.

Target Users

Primary User Profile

- **Who:** A developer or security analyst at the client software company.
- **Need:** Repair a known binary vulnerability and inspect evidence before deciding whether a patched artifact is suitable for further evaluation.
- **Current situation:** Original source is unavailable. Specific current repair tools and workflows are TBD.
- **Expected value:** A repeatable workflow connecting binary analysis, two repair routes, shared validation, and auditable artifacts.
- **Technical level:** Operator expertise is TBD; the first demonstration is operated by the student team with reviewer assistance.

Demonstration Persona

A client analyst supplies the vulnerable ELF, a known failing input, and valid-input tests. A reviewer confirms the repair location and security contract. The analyst starts both repair routes, compares the

resulting evidence, and exports each patched binary with its report. Demographics and commercial adoption targets are not needed for this MVP.

User Journey

1. Open the local browser interface and create a repair job.
2. Select the public ELF and provide vulnerability details, reproducer, valid-input tests, and environment information.
3. Inspect input checks and binary-derived analysis. A reviewer confirms the repair location, intended valid behavior, invalid-input response, and patch constraints.
4. Start both routes from the identical original binary and shared reviewed inputs.
5. Watch route-specific analysis, generation, writeback, and verification progress.
6. Compare test results, bounded verification outcomes, compatibility findings, changed regions, added code, elapsed time, and manual interventions.
7. Export each route's candidate binary and evidence. Unsupported, failed, or inconclusive outcomes remain visible and distinguishable.

The operator must not need terminal commands to complete the demonstration once its environment is prepared. Environment installation may still require technical setup.

MVP Features

All five features below are P0 and required for the agreed demonstration.

F1. Graphical Job Setup and Contract Review

Description: A local browser interface accepts the binary and test details, screens the input, and supports reviewer confirmation before automated repair begins.

User story: As an analyst, I want to define a repair job and review its assumptions so the system repairs the intended defect.

Acceptance criteria:

- The GUI accepts a Linux x86-64 ELF, defect description, reproducer, valid-input tests, and relevant environment information.
- Input identity is recorded with a cryptographic hash.
- Wrong architecture, malformed files, and missing required inputs produce actionable messages before execution.
- The reviewer can see and confirm the repair location and security contract.
- The contract defines the valid-input domain, forbidden behavior, intended invalid-input behavior, and relevant assumptions.
- Both routes reference the same original input hash and reviewed contract.
- Original source is not an input to the repair pipeline.

F2. Direct Instruction Repair

Description: Generate candidate machine-instruction changes and apply them to a separate copy of the binary.

User story: As an analyst, I want an instruction-level repair so I can evaluate a localized patch without original source.

Acceptance criteria:

- Route A uses the binary-derived analysis and reviewed contract.
- Candidate encoding, instruction boundaries, placement, and affected transfers are checked before validation.
- It may use another suitable location when the original location is too small.
- It records edited regions, added code, necessary redirection, and structural changes.
- It does not overwrite the original.
- If safe encoding, placement, redirection, or required compatibility cannot be supported, it reports unsupported with a reason.
- Its final candidate enters the same validation process as Route B.

F3. Recovered C Repair and Reintegration

Description: Recover C-like code for a local region, repair it, compile it, and reintegrate the generated code into a separate copy of the original binary.

User story: As an analyst, I want a C-based repair route so I can express repair logic at a higher level and compare it with instruction repair.

Acceptance criteria:

- Route B begins from recovered C and binary-derived metadata, not original source.
- The unedited recovered output and all manual corrections are retained.
- An unchanged round-trip check precedes security edits; failures are reported instead of silently bypassed.
- Compilation and reintegration account for the generated code's references and interface.
- Placement elsewhere is permitted when needed, subject to the same compatibility and evidence requirements as Route A.
- Unsupported dependencies, interfaces, or placement conditions are explicitly reported.
- The final candidate repairs the same defect in the same original binary used by Route A.
- Compilation success alone is not displayed as successful repair.

Design recommendation, not fixed implementation: start with a small whole function with a clear interface. Exact region granularity and backend belong in technical design.

F4. Shared Validation and Explicit Outcomes

Description: Apply a shared set of structural, compatibility, behavioral, and bounded symbolic checks to each final patched binary.

User story: As a reviewer, I want comparable evidence and clearly stated limitations so I can judge each candidate.

Acceptance criteria:

- Both candidates use the same test inputs, expected outcomes, reviewed contract, and relevant analysis bounds.

- The selected public case demonstrates the specified unsafe behavior before repair.
- Each successful candidate blocks that behavior, meets the invalid-input contract, and passes all agreed valid-input regression tests.
- The actual final patched executable receives the compatibility checks and bounded symbolic security check.
- The symbolic result records its input/memory domain, bounds, assumptions, relevant models, errors, and unresolved exploration.
- Required inconclusive or unrun obligations prevent an overall successful result.
- Failed security or behavioral checks reject the candidate.
- No-crash observations, timeouts, solver unknown, or analysis errors cannot be displayed as proof of correctness.
- Existing agreed tests are included; original behavior is a comparison oracle only on the defined-valid domain.

Terminology: “Security contract” means the reviewed assumptions and required outcomes. “Bounded verification” checks a property within a stated domain/model; it is not an unconditional whole-program guarantee.

F5. Route Comparison and Artifact Export

Description: Present the two outcomes side by side and export replayable evidence.

User story: As an analyst, I want to compare and export both repairs so another reviewer can reproduce the result.

Acceptance criteria:

- The GUI shows both routes for one job, including partial or unsuccessful outcomes.
- Each route shows stage/status, changed and added regions, test counts, verification result, compatibility result, elapsed time, and manual interventions.
- AI usage/cost is shown when available; missing usage information is labeled unavailable.
- Each candidate and its human-readable/machine-readable evidence can be exported with matching input/candidate hashes.
- Failed or inconclusive artifacts are clearly labeled diagnostic candidates.
- An independent teammate can replay the evidence using documented instructions.
- Exports do not imply production approval.

Nice-to-have features: None committed. Additional usability work must not displace a required route or validation obligation.

Out of Scope (Not in MVP)

Item	Boundary or later milestone
Named real-world CVE repair	Next milestone after the public-case MVP
Final production application	Scope and acceptance criteria remain TBD
Arbitrary vulnerability discovery	Initial location and contract are reviewer-confirmed

Item	Boundary or later milestone
Broad architecture/format support	MVP starts with Linux x86-64 ELF
General recovery of original source	Route B recovers only what its local repair requires
Unconditional whole-program correctness proof	Evidence is tied to stated contracts, models, bounds, and tests
Proof of globally smallest possible patch	Minimize unnecessary changes and report size; no global optimum claim
Automatic client deployment/hot patching	Export for review; no production writeback
Hosted processing of confidential client binaries without authorization	Client material stays local unless explicitly authorized
Public SaaS, payments, social features, mobile application	Not required for this local demonstration

Complex or unmodeled ELF features can be reported unsupported. The exact supported executable profile will be specified and tested in technical design; support is not implied by the file format alone.

Success Metrics

Targets are product acceptance requirements, not achieved measurements. All are due at MVP acceptance; the calendar date is TBD.

Metric	Target	Measurement
Shared-case repair	2 successful route outcomes on the same original public binary and defect	Input hashes, route records, contract identifiers
Target prevention	Both candidates meet the reviewed security/invalid-input contract	Reproducer and contract checks on final executables
Valid regression	100% of the agreed valid-input suite passes for each route	Frozen test manifest and pass/fail counts
Bounded verification	Required bounded security obligation satisfied for each final candidate	Assumptions, bounds, model, and solver/exploration evidence
Compatibility	All required checks pass for each candidate	Structural and runtime compatibility records
GUI journey	One complete setup → review → repair → compare → export journey without terminal use	Demonstration checklist
Refusal accuracy	All defined unsupported/error/inconclusive demonstration scenarios receive the correct label	Negative-case results

Metric	Target	Measurement
Reproducibility	One independent teammate replays both route results	Replay log and artifact hashes

Also record modified bytes, added bytes, affected functions/regions, elapsed time, runtime overhead, AI usage, and manual interventions. Numeric performance ceilings are TBD after measurement. Do not claim population-level repair success from one sample.

UI/UX Direction

Design feel: Clear, technical, evidence-focused. This is a working direction, not a requested brand identity.

View	Purpose	Required elements
Job setup and review	Define and confirm the job	File selection, defect/test details, input checks, binary summary, contract and location review
Repair progress and comparison	Track both routes	Independent stage indicators, accessible status labels, explanations, shared job identity
Evidence and export	Inspect results and retrieve artifacts	Test/verification details, assumptions, changed-region comparison, reports, download actions

Interaction requirements:

- Show important limitations beside the relevant result.
- Use text/icons as well as color for statuses.
- Keep technical details expandable while making the main result understandable.
- Make both routes easy to compare without implying that smaller automatically means safer.
- Preserve completed evidence when another stage fails.
- Keep browser navigation responsive during long-running analysis.

Platform: Local browser GUI with Linux x86-64 execution workers. The browser host device may differ from the execution environment. Exact supported browser and OS versions are TBD in Part 3; mobile optimization is not required.

Result Semantics

Outcome	Meaning	Product behavior
Successful within stated scope	All required checks satisfied, including the specified bounded obligation	Export labeled candidate plus scope/assumptions
Unsupported	Required repair/placement/interface capability is outside supported conditions	Explain limitation; do not represent as success
Rejected candidate	A generated candidate fails a required check	Show reason and retain diagnostic evidence

Outcome	Meaning	Product behavior
Inconclusive	Available evidence cannot establish a required property	Show unresolved obligation; do not accept
Error	A tool, provider, or system failed	Show stage/reason; retain completed evidence
Cancelled	Operator stopped the job	Stop further work and label retained evidence appropriately

An overall successful demonstration requires both routes to succeed. One successful route does not satisfy the two-route requirement. Retry policies and exact internal state identifiers are Part 3 decisions.

Technical Considerations

Fixed product requirements: Linux x86-64 ELF; binary-only repair input; both routes; local browser GUI; shared verification; separate patched outputs.

Candidate stack, not approved dependencies: Python coordination; Ghidra; Capstone; Keystone or LLVM assembly; LLVM compilation; LIEF; angr/Claripy/Z3. Patcherex2 and SCRIBE are potential reuse options. Joint compatibility, licensing, version selection, and backend choice require technical validation.

Placement policy: Prefer a localized edit when feasible. Added code and required redirection may be used elsewhere, including necessary mapped-code/metadata changes when supported. Preserve required behavior and compatibility; record each layout change. Unchanged file size is not a universal invariant. If code cannot safely be accommodated, return unsupported.

Quality and performance: Keep long analysis separate from interactive UI work. Provide stage progress and cancellation. Resource budgets, retries, maximum input size, expected runtime, and hardware profile are TBD before implementation acceptance.

Security/privacy: Run binaries in an isolated execution environment; preserve originals; restrict writes to job artifacts. Keep API credentials outside untrusted execution contexts. The exact local-service access protection and isolation implementation are Part 3 decisions.

Scalability: A local demonstration workflow; multi-tenant or enterprise concurrency is not a requirement.

AI / Automation Scope

Aspect	Requirement
AI outcome	Propose repair logic/candidates within the reviewed job scope
Allowed data	Public test-case binaries and approved derived analysis; client artifacts only under explicit client authorization for any external transfer
Client default	Keep future client binaries and derived confidential content local
Allowed actions	Automated candidate generation, patching of copies, and isolated validation after location/contract confirmation

Aspect	Requirement
Output contract	Structured candidate tied to input identity, route, target, expected effects, and uncertainty; exact schema TBD
Independent checks	Deterministic validation controls acceptance; model explanations are not proof
Reviewer confirmation	Repair location and contract before automated runs; external processing of client material requires explicit client authorization
Provider	Hosted API allowed for public cases; provider/model/account, retention settings, and local-model option TBD
Failure behavior	Report errors or inconclusive outcomes; never fabricate a candidate or positive verification
Logging	Preserve audit records; exclude credentials; exact retention policy TBD
Cost controls	Usage visibility and configurable limits; numeric ceilings TBD

Required evaluation scenarios: valid public-case request; malformed candidate; wrong architecture or input identity; insufficient safe placement; failing regression; solver timeout/unknown; provider failure; embedded strings attempting to change tool permissions; client-data transfer attempted without authorization; unsuccessful retry sequence; cancellation.

A deterministic repair baseline is a research recommendation for evaluation, not a replacement for the agreed AI-assisted product capability.

Constraints & Requirements

Area	Confirmed constraint	Still TBD
Team	Five first-year students; basic Python only	Weekly available hours and named owners
Budget	Flexible	Actual API/compute limits and approved purchases
Proposal	Due in one week from 15 October 2026	Exact submission time and required format
MVP	Public stack-buffer-overflow case; both routes, same binary	Exact sample, build, environment, deadline
Next milestone	Named real-world CVE	Which CVE and acceptance date
Final app	Later delivery stage	Supported corpus, workflows, deployment, acceptance
Source boundary	Original source unavailable to repair pipeline	How evaluator-only source is separated in selected fixture

Area	Confirmed constraint	Still TBD
Verification	Agreed tests, compatibility, bounded symbolic checks	Exact contract, bounds, oracle, and test manifest

Budget figures in research are estimates, not committed spending. Timeline estimates are not delivery promises.

Open Questions & Assumptions

Blocking technical-design decisions:

- Select the public sample and verify that both routes can repair it.
- Freeze valid/invalid inputs, expected results, contract, and test manifest.
- Define supported ELF features and function interfaces.
- Prove feasibility of the unchanged recovered-C round trip.
- Choose placement/redirection backend and layout compatibility checks.
- Specify the symbolic model/domain and conditions for accepting its result.
- Choose GUI framework, service/worker design, isolation, browser support, and hardware.
- Choose AI provider, structured schema, retention policy, retry/time/cost limits.
- Confirm MVP/final dates, weekly capacity, and exact client acceptance process.

Planning assumptions, not established facts: a suitable public case exists for the two-route demonstration; reviewers can approve its contract; the selected toolchain can be made compatible. If a feasibility experiment disproves these assumptions, report the blocker and seek a scope change rather than silently omitting a route.

Risk Mitigation

Risk	Impact	Mitigation
Recovered C compiles but changes behavior	High	Unchanged round-trip gate before repair
New code breaks addressing or calling conventions	High	Placement audit and shared compatibility checks
A patch suppresses useful behavior	High	Positive regression suite and independent invalid-input contract
Symbolic model omits relevant paths	High	Explicit assumptions/state accounting; inconclusive status
One route works but the other does not	High	Two-route feasibility gate before schedule commitment
GUI masks backend failure	High	Real per-stage data and explicit terminal outcomes
Beginner workload exceeds capacity	High	Shared foundations, staged milestones, early blockers, re-estimation

MVP Completion Checklist

- [ ] F1–F5 implemented with observable acceptance evidence.
- [ ] One selected public binary and vulnerability used for both routes.
- [ ] Reviewer-confirmed contract and location recorded.
- [ ] Route A produces its own patched copy.
- [ ] Route B uses recovered C and passes its round-trip prerequisite.
- [ ] Both final candidates meet all shared required checks.
- [ ] Unsupported, rejected, inconclusive, error, and cancelled cases remain distinct.
- [ ] Graphical demonstration runs end to end without operator terminal use.
- [ ] Changed/added regions and relevant layout changes are disclosed.
- [ ] Human-readable and machine-readable reports export correctly.
- [ ] Public/client data boundaries and AI failure scenarios are tested.
- [ ] Original binary preserved; artifacts traceable by hashes.
- [ ] Independent teammate replays both results.
- [ ] No universal proof, global-minimality, or production-readiness claim appears.
- [ ] User/setup and evidence-replay instructions are available.

Next Steps

1. Use Part 3 to resolve the listed technical decisions.
2. Run feasibility experiments before committing an MVP schedule.
3. Prepare the proposal using confirmed scope and honestly labeled evidence.
4. Implement and validate the public-case GUI demonstration.
5. After MVP acceptance, select a real-world CVE for the next milestone.

PRD version: 1.0
Product brief confirmed: 15 October 2026
Status: Ready for technical design; implementation feasibility remains to be tested
Owner: Five-person student team
Stakeholders: Student team and client software company

Handoff Context

- Stage: prd
- App name: BinaryRepair
- Full project title: AI-Driven Automated Binary Vulnerability Repair and Correctness Verification
- User level: C — five first-year university students; basic Python only
- Target platform: Linux x86-64 ELF execution; local browser graphical interface
- Budget: flexible; numeric operating limits TBD
- Timeline: proposal one week from 15 October 2026, approximately 22 October, exact time TBD; MVP and final-app deadlines TBD
- AI in product scope: yes; hosted API permitted for public cases
- Research status: Researched with sources; documentation/literature only; no build, repair, benchmark, or verification experiment performed
- Constraints: binary-only repair workflow; same public stack-buffer-overflow vulnerability and same original ELF for both routes; reviewer-confirmed location and contract; preserve required behavior and compatibility

- Decisions: both instruction and recovered-C routes mandatory; graphical MVP via local browser; code elsewhere allowed with recorded layout changes; unsupported if safe accommodation/redirection impossible; shared tests and bounded verification; separate patched artifacts; real-world CVE next milestone
- AI data boundary: public samples may use hosted APIs; future client binaries and confidential derived content stay local unless the client explicitly authorizes external sharing
- Success criteria: both route candidates meet security/invalid-input contract, pass all agreed valid-input tests and compatibility checks, satisfy required bounded security verification, and export replayable evidence through the GUI
- Out of scope: first-MVP real-world CVE, final production app, arbitrary discovery, broad format/architecture support, general original-source recovery, unconditional whole-program proof, global minimality proof, production deployment, unauthorized confidential-data transfer, public SaaS/payments/mobile app
- Open questions: exact sample/build/contract/tests; supported ELF profile; tool/backend/framework choices; symbolic model/bounds; AI provider/retention/limits; hardware/browser/isolation; later deadlines; final-app scope
- Source files: research-BinaryRepair.md → docs/PRD-BinaryRepair-MVP.md
- Decision precedence: confirmed Part 2 decisions override earlier research recommendations for CLI, strict in-place-only policy, and unresolved same-sample scope


```
{
  "schemaVersion": 1,
  "documentType": "prd",
  "appName": "BinaryRepair",
  "oneLiner": "Repair one known public stack-buffer-overflow vulnerability
in the same Linux x86-64 ELF through two AI-assisted routes and compare
validation evidence in a local browser GUI.",
  "targetUsers": "Software-company developers and security analysts without
original source code; student team operates the first demonstration with
reviewer assistance.",
  "phase": "Foundation",
  "mustHave": [
    "F1: Graphical job setup and contract review",
    "F2: Direct instruction repair",
    "F3: Recovered C repair and reintegration",
    "F4: Shared validation and explicit outcomes",
    "F5: Route comparison and artifact export"
  ],
  "niceToHave": [],
  "notInMvp": [
    "Named real-world CVE repair; next milestone",
    "Final production application",
    "Arbitrary vulnerability discovery",
    "Broad architecture and binary-format support",
    "General recovery of original source",
    "Unconditional whole-program correctness proof",
    "Proof of globally smallest patch",
    "Automatic production deployment or hot patching",
    "Unauthorized external processing of confidential client binaries",
    "Public SaaS, payments, social features, or mobile application"
```

```
  ],
  "successMetrics": [
    "Two successful route outcomes on the same original public binary and
defect",
    "Both candidates meet the reviewed security and invalid-input
contract",
    "Each route passes 100% of the agreed valid-input regression suite",
    "Each final candidate satisfies the required bounded security
obligation",
    "Each final candidate passes all required compatibility checks",
    "One complete GUI journey without operator terminal use",
    "All defined negative demonstration scenarios receive correct outcome
labels",
    "One independent teammate replays both route results"
  ]
}
```